

Guía de estudio ilustrada (v2)

Administración de memoria: paginación y segmentación

Basada en el documento de la Unidad 1 – Sistemas Operativos 2

Idea clave (subrayar)

Todo el documento gira alrededor de **tres funciones** que el sistema operativo debe cumplir sobre la memoria:

1. **Relocalización:** mover un programa de lugar en memoria sin que deje de funcionar.
2. **Protección:** que un proceso no pueda leer/escribir memoria de otro.
3. **Compartición:** que dos procesos puedan usar el mismo bloque cuando convenga.

Cada mecanismo del curso se evalúa contra estas tres funciones. Anotación útil al margen de cada tema: “¿este mecanismo cumple las 3?”

Índice

1. Conceptos básicos antes de empezar	2
2. Evolución de los mecanismos de memoria	3
2.1. Carga estática	3
2.2. Carga dinámica: registros de relocalización/límite	4
3. Direcciones y tamaño de palabra	5
4. Paginación	5
4.1. Los tres componentes del mecanismo	6
4.2. El Page Table Pointer y el cambio de contexto	6
4.3. Dos formas de calcular la dirección física	6
4.4. Ejemplo completo (Figura 3 del documento)	7
5. Enlace estático y dinámico	8
6. Segmentación	10
7. Sistemas híbridos (segmentación + paginación)	11
8. Paginación multinivel	12

1. Conceptos básicos antes de empezar

Antes de entrar al documento, aclaremos el vocabulario que se usa en todas las páginas. Si estos conceptos ya los dominas, salta directo a la sección 2.

Concepto básico: bit, byte y palabra

Un **bit** es la unidad mínima de información: un 0 o un 1. Un **byte** son 8 bits. Una **palabra** (*word*) es el grupo de bits que el procesador mueve “de un jalón” entre la memoria y sus registros: 8, 16, 32 o 64 bits según la máquina. Cuando decimos “sistema de 64 bits” hablamos del tamaño de palabra.

Concepto básico: CPU y registros

La **CPU** (procesador) es el chip que ejecuta las instrucciones. Un **registro** es una celda de memoria *ultra rápida que vive dentro de la CPU* (no en la RAM); el procesador solo puede operar con datos que estén en registros. Ejemplos en Intel: **AX** (acumulador), **IP** (instruction pointer: qué instrucción sigue), **CS/DS/SS** (registros de segmento para código, datos y pila). El MMU también tiene sus propios registros (límite, relocalizador, page table pointer...).

Concepto básico: memoria RAM y direcciones

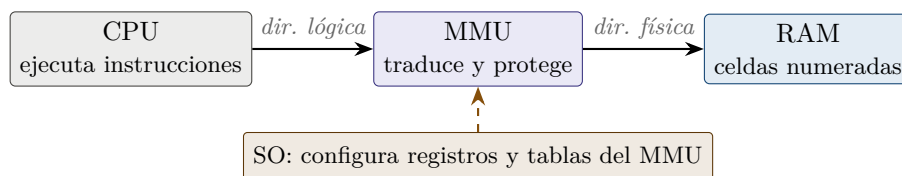
La **RAM** (memoria principal) es una enorme fila de celdas numeradas; el número de cada celda es su **dirección**. Leer o escribir memoria siempre es “dame/guarda el contenido de la dirección *X*”. Una dirección puede referirse a un byte o a una palabra, según el diseño del sistema.

Concepto básico: dirección lógica vs. dirección física

La **dirección física** es el número real de la celda en el chip de RAM. La **dirección lógica** (o *virtual*) es la que el programa *crea* estar usando: cada proceso ve su memoria como si empezara en 0 y fuera continua. Alguien tiene que traducir entre ambas en cada acceso... y ese alguien es el MMU.

Concepto básico: MMU (Memory Management Unit)

Es un **componente de hardware** (hoy integrado dentro del procesador) que se sienta entre la CPU y la memoria: *toda* dirección que la CPU emite pasa por el MMU, que la traduce de lógica a física y verifica permisos. Es hardware y no software por velocidad: la traducción ocurre en **cada** acceso a memoria, miles de millones de veces por segundo; hacerlo por software sería lentísimo. El SO *configura* al MMU (le carga los registros y las tablas), pero el MMU hace el trabajo en cada acceso.



Concepto básico: proceso y cambio de contexto

Un **proceso** es un programa en ejecución: su código + sus datos + el estado de todos sus registros. Como hay un solo procesador (o pocos) y muchos procesos, el SO los turna. Un **cambio de contexto** es el momento en que el SO quita la CPU a un proceso y se la da a otro: guarda todos los registros del que sale y **restaura** los del que entra — incluyendo los registros del MMU (límite/relocalizador o el page table pointer). Por eso cada proceso “ve” su propia memoria.

Concepto básico: sistema operativo y kernel

El **sistema operativo (SO)** es el software que administra el hardware y da servicios a los programas. Su núcleo, el **kernel**, es la parte que siempre está en memoria y tiene todos los privilegios: administra memoria, procesos, archivos y dispositivos. Cuando el documento dice “la memoria del kernel está en la memoria alta”, significa que el SO se coloca en las direcciones físicas superiores, dejando las bajas (desde 0) para los marcos de página de los procesos.

Concepto básico: DOS

DOS (*Disk Operating System*, el más famoso: MS-DOS de Microsoft, años 80) fue un SO **mono-usuario y monotarea**: un solo programa a la vez, sin protección de memoria. Sus ejecutables **.COM** eran imágenes crudas del programa que se cargaban en una dirección fija — el ejemplo perfecto de *carga estática*. Los **.EXE** posteriores ya incluían información para relocalizar.

Concepto básico: ejecutable (.exe) y librería (.dll)

Un **ejecutable (.exe)** es el archivo con el programa listo para correr. Una **librería** es código reutilizable (funciones matemáticas, gráficas, de red...) que muchos programas necesitan. **DLL** significa *Dynamic Link Library* (librería de enlace dinámico, **.dll** en Windows; en Linux son los **.so**, *shared object*). El **compilador** es el programa que traduce código fuente (C, C++...) a código máquina, y el **enlazador** (*linker*) une las piezas: ahí se decide si el enlace es estático o dinámico.

Concepto básico: hexadecimal

Los números como FF, B1 o EF que aparecen en el documento están en base 16 (**hexadecimal**): dígitos 0–9 y A–F (A=10 ... F=15). Se usa porque cada dígito hex equivale exactamente a 4 bits: $FF = 11111111_2 = 255$. Es la forma compacta estándar de escribir direcciones de memoria.

2. Evolución de los mecanismos de memoria

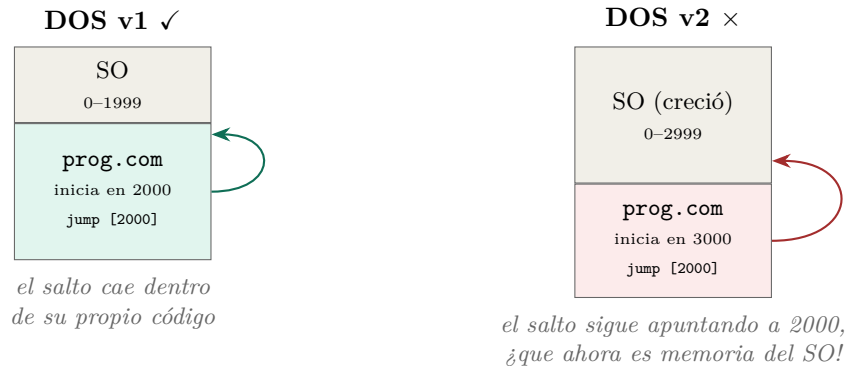
El hardware de administración de memoria evolucionó en tres fases: **carga estática** → **carga dinámica** (registros de relocalización/límite) → **paginación**.

2.1. Carga estática

En los primeros sistemas (monousuario, como DOS) no había hardware de protección. El programador debía conocer *de antemano* la dirección física donde se cargaría el programa, y escribía **referencias absolutas** tanto para datos (lecturas) como para código (saltos).

Ejemplo desarrollado: los .COM de DOS

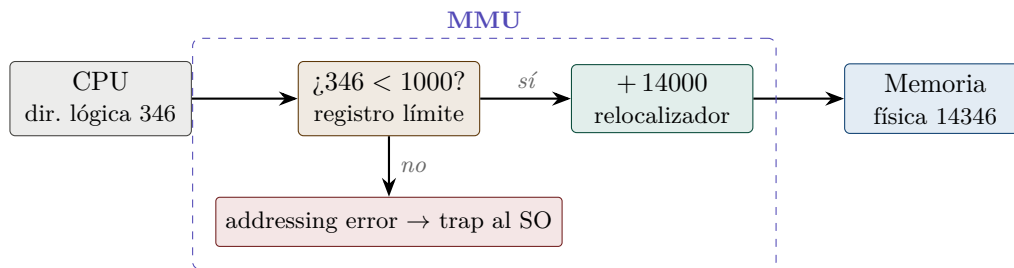
Los ejecutables .COM estaban programados para iniciar en una posición fija, que dependía de cuánta memoria ocupaba esa versión de DOS. Al cambiar de versión, el SO ocupaba distinta cantidad de memoria, la posición inicial disponible cambiaba y los programas de la versión anterior **dejaban de funcionar**: sus saltos absolutos caían en memoria equivocada.



2.2. Carga dinámica: registros de relocación/límite

La solución: el programa usa **direcciones relativas** (su memoria “empieza en 0”) y el MMU traduce en cada acceso usando dos registros:

- **Registro límite**: cuánta memoria tiene asignada el proceso. Si la dirección lógica $\geq$ límite $\Rightarrow$ *addressing error* (protección): el MMU avisa al SO, que típicamente termina al proceso.
- **Registro relocizador (base)**: dónde empieza realmente el proceso en memoria física. Se *suma* a la dirección lógica (relocalización).



Idea clave (subrayar)

El límite y el relocizador **forman parte del contexto del proceso**: en cada cambio de contexto el SO debe restaurarlos con los valores del proceso que entra a ejecutar, igual que sus demás registros. Si el SO quiere mover el proceso, solo cambia el relocizador; el programa ni se entera.

Ejemplo desarrollado: registros base en los primeros Intel

La relocación se hacía con registros de segmento (DS datos, CS código, SS pila). Las instrucciones

```
mov ax, [FF]      jump [5]
```

equivalen implícitamente a

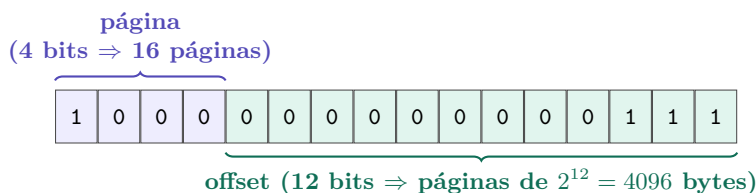
```
mov ax, DS:[FF]   jump CS:[5]
```

Es decir: “mueve a AX el contenido de la dirección FF *relativa al inicio de mis datos*” y “salta a la instrucción 5 *relativa al inicio de mi código*”. El programador solo piensa en direcciones relativas.

3. Direcciones y tamaño de palabra

- **Tamaño de palabra:** número de bits que se transfieren de memoria a un registro en una operación (el ancho del bus del sistema).
- Una **dirección** puede referirse a un byte, a un grupo de bytes o a una palabra.

Esto importa porque en paginación la dirección se *parte en campos de bits*. Con n bits de dirección se pueden formar 2^n direcciones distintas. Ejemplo con 16 bits, usando 4 para página y 12 para offset:

**Idea clave (subrayar)**

El número de bits del offset **determina el tamaño de página** (2^{bits}), y los bits restantes determinan **cuántas páginas** puede tener el espacio lógico. Repartir los bits de la dirección *es* diseñar el esquema de paginación.

4. Paginación

Definición: paginación

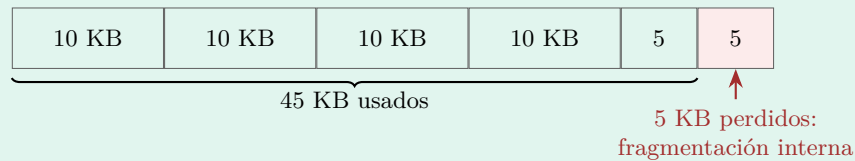
Dividir la memoria en partes **iguales y de tamaño fijo**. Del lado lógico se llaman **páginas**; del lado físico, **marcos de página** (frames). La página es la *unidad mínima de asignación*: toda petición de memoria se redondea hacia arriba a páginas completas.

Definición: fragmentación interna

Desperdicio de memoria *dentro* de un bloque de tamaño N entregado a un proceso que pidió $K < N$: se pierden $N - K$ dentro del bloque, y nadie más puede usarlos.

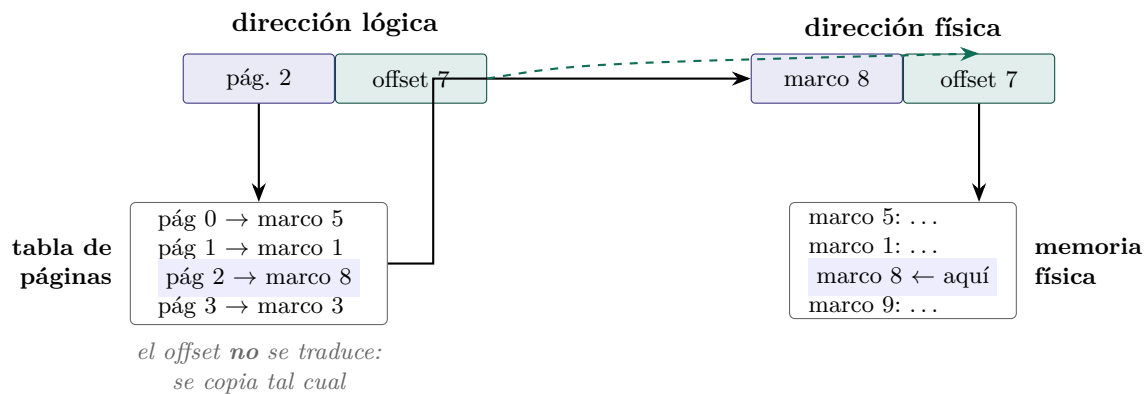
Ejemplo desarrollado: fragmentación interna

Páginas de 10 KB, el proceso pide 45 KB $\Rightarrow$ el SO entrega $\lceil 45/10 \rceil = 5$ páginas = 50 KB. En la última página quedan $50 - 45 = 5$ KB desperdiciados:



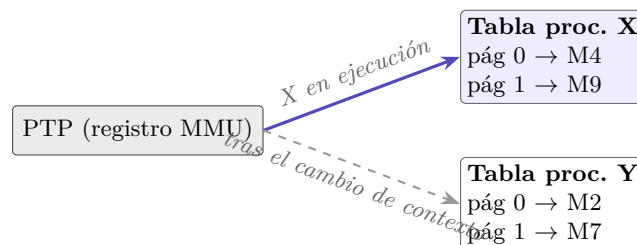
4.1. Los tres componentes del mecanismo

1. **Dirección lógica dividida en dos partes:** número de página (bits altos) + desplazamiento u *offset* (bits bajos).
2. **Tabla de páginas** (una por proceso): para cada página lógica indica en qué marco físico está, además de PID propietario y permisos.
3. **Page Table Pointer (PTP):** registro del MMU que apunta a la tabla de páginas del proceso *en ejecución*; se restaura en cada cambio de contexto, junto con IP, DS, SS, etc.



4.2. El Page Table Pointer y el cambio de contexto

Cada proceso tiene *su propia* tabla de páginas. El MMU solo mira la tabla a la que apunta el PTP; cambiar de proceso = cambiar el PTP:



4.3. Dos formas de calcular la dirección física

- 1) **Por suma.** La tabla guarda la *dirección física completa* del marco:

$$dirFísica = frameDir + offset$$

Desventajas: entradas más grandes en la tabla y una suma por cada acceso.

2) Por yuxtaposición. La tabla guarda solo el *número* de marco:

$$dirFísica = dirBase + frameNumber \cdot pageSize + offset$$

Como $pageSize = 2^{bits}$, la multiplicación se sustituye por un corrimiento de bits ($frameNumber \ll bits$), muchísimo más eficiente en microoperaciones. Además, usualmente los marcos inician en la dirección física 0 (el kernel vive en memoria alta), así que $dirBase = 0$: el resultado es literalmente *pegar* los bits del marco con los del offset.

Ejemplo desarrollado: yuxtaposición con números

Páginas de 4 KB $\Rightarrow$ offset de 12 bits. Marco = 8 = 1000₂, offset = 7 = 000000000111₂.

$$\underbrace{1000}_{\text{marco}} \underbrace{000000000111}_{\text{offset (12 bits)}} = 1000000000000111_2 = 32775$$

Comprobación: $8 \times 4096 + 7 = 32768 + 7 = 32775$. ✓ No hubo suma real: solo se *concatenaron* los bits.

Cuidado / típica pregunta de examen

Balance del tamaño de página (se configura en el MMU al iniciar el SO):

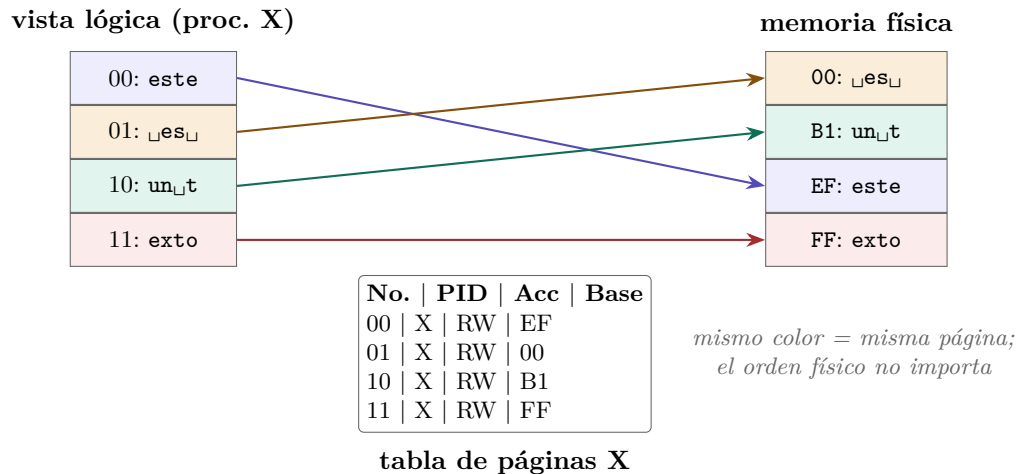
- Páginas **grandes** $\rightarrow$ tabla pequeña ✓, pero más fragmentación interna ×.
- Páginas **pequeñas** $\rightarrow$ poco desperdicio ✓, pero tablas enormes y búsquedas más costosas ×.

4.4. Ejemplo completo (Figura 3 del documento)

Sistema teórico: direcciones de **4 bits** (máximo $2^4 = 16$ direcciones), palabra de 8 bits. Se usan **2 bits para número de página** y **2 bits para offset** $\Rightarrow$ 4 páginas de 4 direcciones cada una. El proceso X guarda el string “*este es un texto*” (16 caracteres, uno por dirección).

Dirección lógica		Página	Contenido
binario	decimal		
0000–0011	0–3	00	e s t e
0100–0111	4–7	01	▯ e s ▯
1000–1011	8–11	10	u n ▯ t
1100–1111	12–15	11	e x t o

La tabla de páginas del proceso X guarda, por cada página: **PID propietario**, **permisos** y **dirección base** del marco. Lo que para el proceso es continuo, en la memoria física está **regado en cualquier orden**:

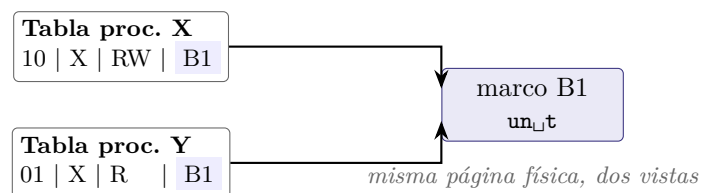


Idea clave (subrayar)

Si el SO mueve una página física de lugar, solo actualiza *una entrada* de la tabla: el proceso sigue viendo las mismas direcciones lógicas (0000 a 1111) sin ningún cambio. Eso es la **relocalización** en paginación.

Compartición entre procesos. Si un proceso Y necesita usar la página 10 del proceso X, X la comparte *explícitamente* y en la tabla de Y se agrega una entrada apuntando al *mismo marco* (B1). Dos observaciones del documento (típicas de examen):

1. La **dirección lógica no es la misma** en ambos procesos: cada tabla asigna la entrada según su propio estado previo. X la ve como su página 10; Y puede verla como su página 01.
2. La entrada en la tabla de Y indica que el **propietario es X** y que Y tiene **solo lectura (R)**. Eso es la **protección** aplicada a la **compartición**.



5. Enlace estático y dinámico

Caso real donde brilla la compartición por paginación: las **librerías**.

- **Enlace estático:** el ejecutable contiene *dentro de sí* todas las librerías que usa. Distribución simple (un solo archivo), pero si dos programas usan la librería A, ésta queda cargada **dos veces** en memoria: desperdicio de memoria y de tiempo de carga.
- **Enlace dinámico:** el ejecutable contiene solo su código propio; las librerías viven en archivos aparte (A.dll, B.dll, C.dll) que se cargan *una sola vez* y se comparten entre programas.



Secuencia de carga (ejemplo del documento). Al ejecutar X: pide la librería A → no está en memoria → el SO la carga y le comparte su dirección; luego lo mismo con C. Al ejecutar Y: pide A → **ya está cargada** → el SO solo le comparte la dirección; pide B → se carga.

Costo del enlace dinámico. Mantenimiento más complejo: las nuevas versiones de una librería deben mantener **compatibilidad hacia atrás** (en interfaz y funcionamiento) para no romper a los programas que la usan.

Código típico (lo que el compilador genera por nosotros). El programador escribe algo como:

```
int a1(int a, X c); extern "a.dll"; ... int z = a1(0, null);
```

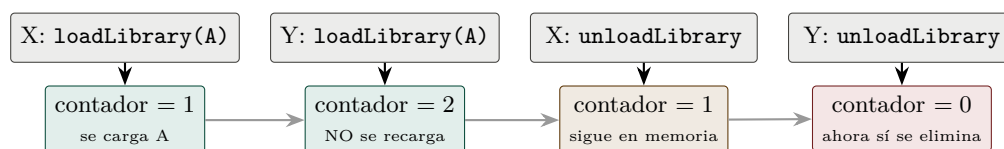
y el compilador lo transforma en llamadas explícitas al SO:

```
handle = loadLibrary("a.dll"); a1 = getProc(handle, "a1"); z = a1(0, null); unloadLibrary(handle);
```

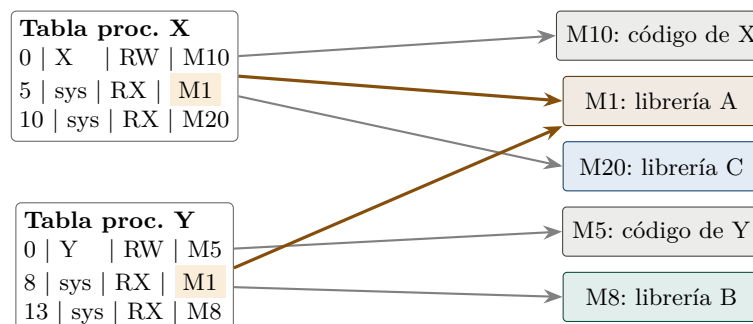
donde `loadLibrary` pide al SO cargar la librería (devuelve un *handle*, un identificador), `getProc` obtiene la dirección de una función dentro de ella, y `unloadLibrary` avisa que ya no se usará.

Definición: conteo de referencias

`unloadLibrary` **no** elimina la librería de memoria: el SO lleva un *contador de procesos* que la usan. Solo cuando el contador llega a 0 puede eliminarla sin afectar a nadie.



Las librerías en las tablas de páginas (Figura 7). Cada proceso tiene entradas para su código propio (propietario él mismo, RW) y entradas para las páginas de las librerías, cuyo **propietario es el sistema (sys)** con permisos **RX** (leer y ejecutar, nunca escribir). Ambos procesos apuntan al *mismo marco* de la librería común:



*M1 (lib A) es compartido:
dos tablas → un marco*

6. Segmentación

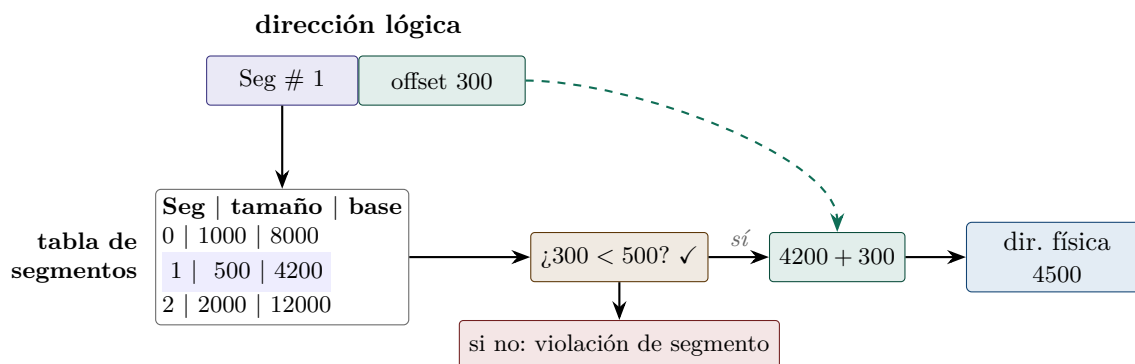
La paginación cumple las tres funciones, pero sufre **fragmentación interna** y además el programador piensa la memoria como *bloques semánticos* (el bloque de código, el de datos, el de la pila...), no como páginas uniformes.

Definición: segmentación

Mismo principio que la paginación, pero con bloques **semánticamente relacionados y de tamaño variable**: cada segmento mide exactamente lo que el proceso pidió. Eso **elimina la fragmentación interna**.

Componentes (análogos a la paginación): dirección lógica = (número de segmento, offset); **tabla de segmentos** con dirección base y *tamaño* de cada segmento; **Segment Table Pointer**.

Mecanismo de traducción: (1) buscar el Seg # en la tabla; (2) verificar que *offset* < *tamaño* del segmento — aquí vive la protección: el límite ahora es *por segmento*; (3) *dirFísica* = *base* + *offset*. Aquí **siempre hay suma**: como los segmentos son de tamaño variable, no se puede yuxtaponer bits.

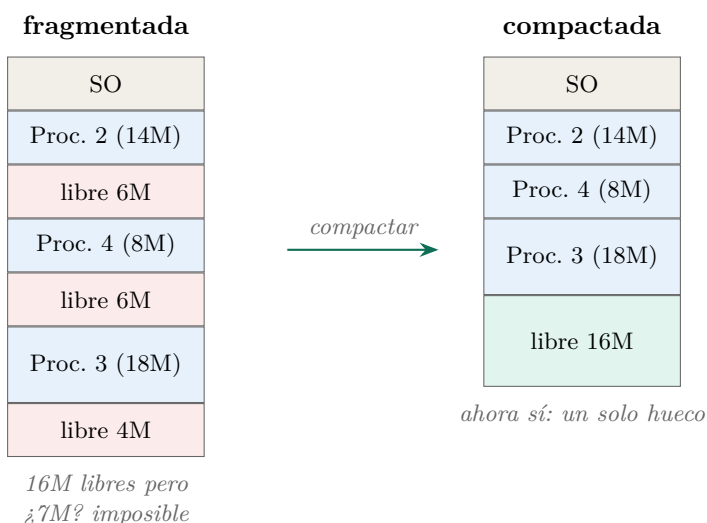


Definición: fragmentación externa

Pequeños bloques *libres* intercalados entre bloques ocupados: aunque la suma total libre sea N , el sistema no puede atender solicitudes de tamaño N (ni menores que N pero mayores que el hueco más grande), porque la memoria libre está **dispersa**.

Ejemplo desarrollado: fragmentación externa (Figura 9)

Procesos 2, 3 y 4 ocupan 14, 18 y 18 MB. Quedan libres tres huecos: 6, 6 y 4 MB = **16 MB libres en total**. Llega una solicitud de **7 MB** y... no se puede atender: ningún hueco individual alcanza.



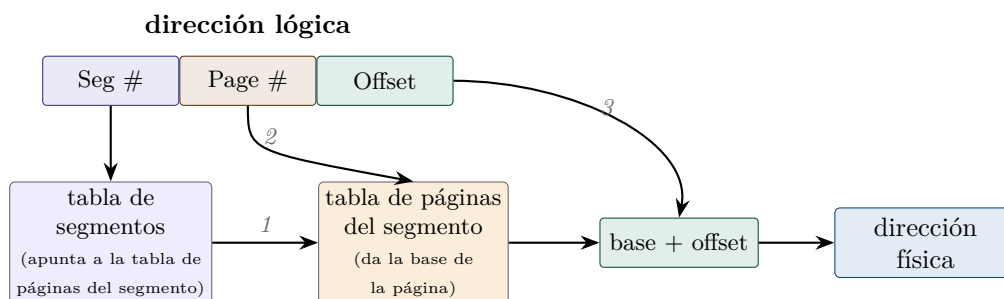
Cuidado / típica pregunta de examen

La **compactación** resuelve la fragmentación externa moviendo todos los bloques ocupados a un extremo... pero mientras se ejecuta, **los procesos de usuario y del sistema deben detenerse** hasta terminar de relocalizar todo. Impacto directo al rendimiento. (Regla nemotécnica: paginación → fragmentación **interna**; segmentación → fragmentación **externa**.)

7. Sistemas híbridos (segmentación + paginación)

La segmentación, además de la fragmentación externa, complica el control por la *variabilidad de tamaños* (como veremos al estudiar memoria virtual). Los sistemas híbridos toman lo mejor de ambos mundos: **primer nivel de segmentos, segundo nivel de paginación**. El programador pide segmentos (visión semántica); transparentemente cada segmento se divide en páginas (asignación física uniforme).

La dirección lógica ahora tiene **tres partes**, y la traducción encadena dos tablas:



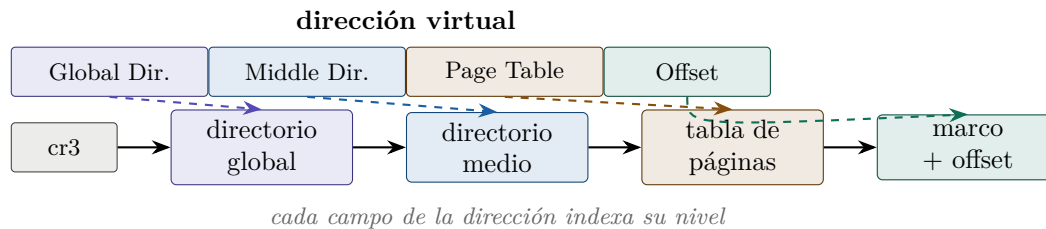
Paso a paso: (1) el Seg # se busca en la tabla de segmentos, que ya **no** contiene la dirección del segmento sino la dirección de *la tabla de páginas de ese segmento*; (2) el Page # se busca en esa tabla de páginas → dirección base de la página; (3) se suma el offset → dirección física.

Idea clave (subrayar)

Si hay fragmentación interna, queda limitada a **la última página de cada segmento**. Y no hay fragmentación externa, porque físicamente todo se asigna en páginas iguales. Cada esquema cubre la debilidad del otro.

8. Paginación multinivel

El concepto de dos niveles se generaliza a una **jerarquía de tablas de páginas**: la dirección virtual se parte en varios índices y un registro (como **cr3** en x86) apunta a la tabla raíz del proceso. En cada nivel, el campo correspondiente de la dirección selecciona una entrada que apunta a la tabla del siguiente nivel.



Idea clave (subrayar)

La ventaja: las tablas de niveles inferiores se crean **dinámicamente, solo cuando se necesitan**. Si un proceso usa apenas una fracción de su espacio de direcciones (el caso normal), no se gasta memoria en tablas para las regiones que nunca toca. Con una tabla única y plana habría que reservar la tabla completa aunque el proceso use el 1 % de sus direcciones.

Resumen comparativo

	Reg. base/límite	Paginación	Segmentación
Unidad de asignación	bloque continuo	página (fija)	segmento (variable)
Relocalización	cambiar registro base	actualizar tabla de páginas	actualizar tabla de segmentos
Protección	registro límite	PID + permisos por página	tamaño por segmento
Compartición	no práctica	entradas al mismo marco	entradas al mismo segmento
Cálculo de dirección	suma	yuxtaposición (o suma)	siempre suma
Problema principal	todo el proceso continuo	fragmentación interna	fragmentación externa
Solución evolutiva	paginación	páginas pequeñas / multinivel	híbrido seg.+pag. / compactación

Cuidado / típica pregunta de examen

Preguntas rápidas para autoevaluarte con esta guía:

1. ¿Por qué los .COM de DOS dejaban de funcionar al cambiar de versión?
2. ¿Qué dos verificaciones/operaciones hace el MMU con base y límite?
3. ¿Por qué la yuxtaposición es más eficiente que la suma? ¿Por qué en segmentación no se puede usar?
4. ¿Por qué la dirección lógica de una página compartida puede ser distinta en cada proceso?
5. ¿Qué pasa cuando el contador de referencias de una DLL llega a 0?
6. ¿Qué fragmentación produce cada esquema y cómo la resuelve el híbrido?